



ALFIDO TECH

RIDING THE WAVES OF TECHNOLOGICAL
ADVANCEMENT

HOUSE PRICE PREDICTION USING MACHINE LEARNING

Alfido Tech Machine Learning Internship

Submitted By

Name: Ankit Bungla

Internship: Machine Learning Internship

Organization: Alfido Tech

Project: House Price Prediction using Machine Learning

Submission Date: 20-07-2026

Table of Contents

1. Introduction

2. Project Objective

3. Dataset Description

4. Exploratory Data Analysis (EDA)

5. Data Preprocessing

6. Feature Engineering

7. Machine Learning Models

8. Model Evaluation

9. Residual Analysis

10. Prediction Example

11. Conclusion

12. References

1. Introduction

House price prediction is one of the most common regression problems in machine learning. Accurate prediction of property prices helps buyers, sellers, and real estate companies make informed decisions.

The objective of this project was to build a machine learning regression model capable of predicting house prices using various housing features. Multiple regression algorithms were trained and compared to identify the best-performing model.

2. Project Objective

The primary objectives of this project were:

- Build a regression model for house price prediction.
- Perform Exploratory Data Analysis (EDA).
- Handle missing values and data preprocessing.
- Apply feature engineering techniques.
- Compare multiple machine learning models.
- Evaluate models using RMSE and MAE.
- Perform residual analysis.
- Save the best model for future predictions.

3. Dataset Description

The dataset contains residential house information with approximately **4,600 records** and **18 features**.

Target Variable

Price

Important Features

- Bedrooms
- Bathrooms
- Square Feet Living Area
- Square Feet Lot
- Floors
- Waterfront
- View
- Condition
- Square Feet Above
- Square Feet Basement
- Year Built
- Year Renovated
- Street
- City

- StateZip
- Country

The dataset contains both numerical and categorical features, making preprocessing an important step before model training.

```
print("Dataset Shape:", df.shape)

df.head()
```

Dataset Shape: (4600, 18)

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	yr_built	yr_renovated	street	city	statezip	country
0	2014-05-02 00:00:00	313000.0	3.0	1.50	1340	7912	1.5	0	0	3	1340	0	1955	2005	18810 Densmore Ave N	Shoreline	WA 98133	USA
1	2014-05-02 00:00:00	2384000.0	5.0	2.50	3650	9050	2.0	0	4	5	3370	280	1921	0	709 W Blaine St	Seattle	WA 98119	USA
2	2014-05-02 00:00:00	342000.0	3.0	2.00	1930	11947	1.0	0	0	4	1930	0	1966	0	26206-26214 143rd Ave SE	Kent	WA 98042	USA
3	2014-05-02 00:00:00	420000.0	3.0	2.25	2000	8030	1.0	0	0	4	1000	1000	1963	0	857 170th Pl NE	Bellevue	WA 98008	USA
4	2014-05-02 00:00:00	550000.0	4.0	2.50	1940	10500	1.0	0	0	4	1140	800	1976	1992	9105 170th Ave NE	Redmond	WA 98052	USA

Next steps: [Generate code with df](#) [New interactive sheet](#)

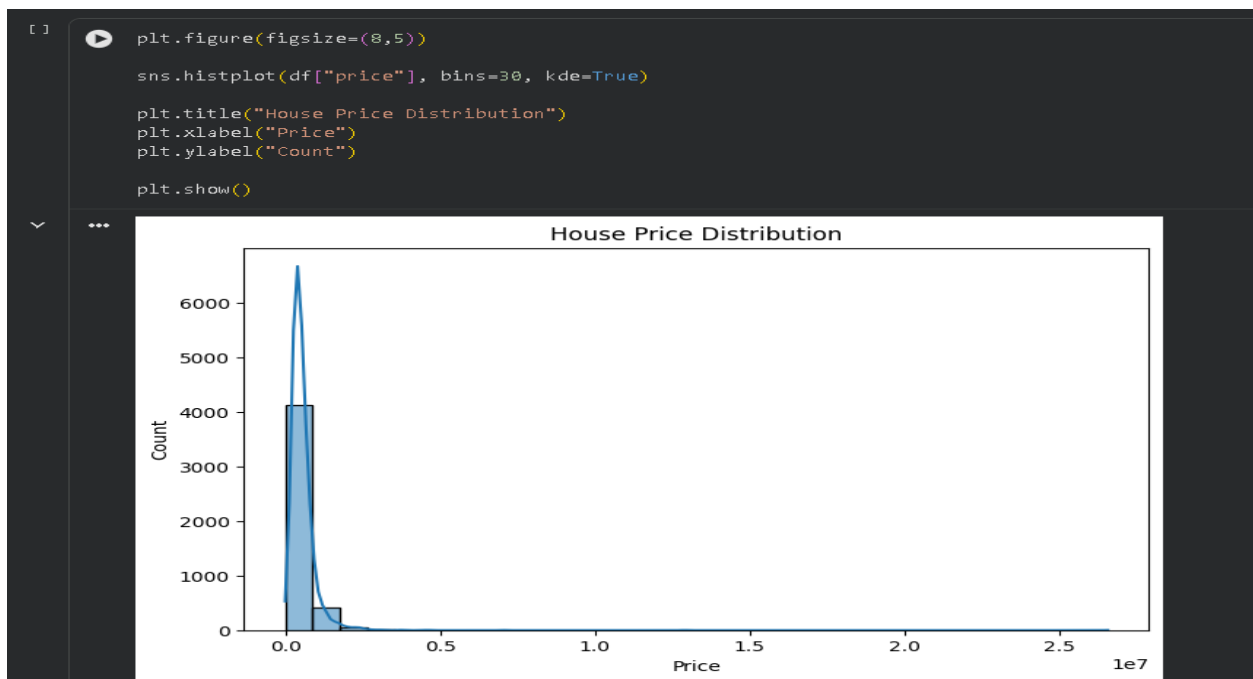
4. Exploratory Data Analysis (EDA)

EDA was performed to understand the structure and quality of the dataset.

The following analyses were conducted:

- Dataset information
- Missing value analysis
- Statistical summary
- Distribution of house prices
- Correlation heatmap
- Feature relationships

These visualizations helped identify feature importance and understand the relationships between variables.

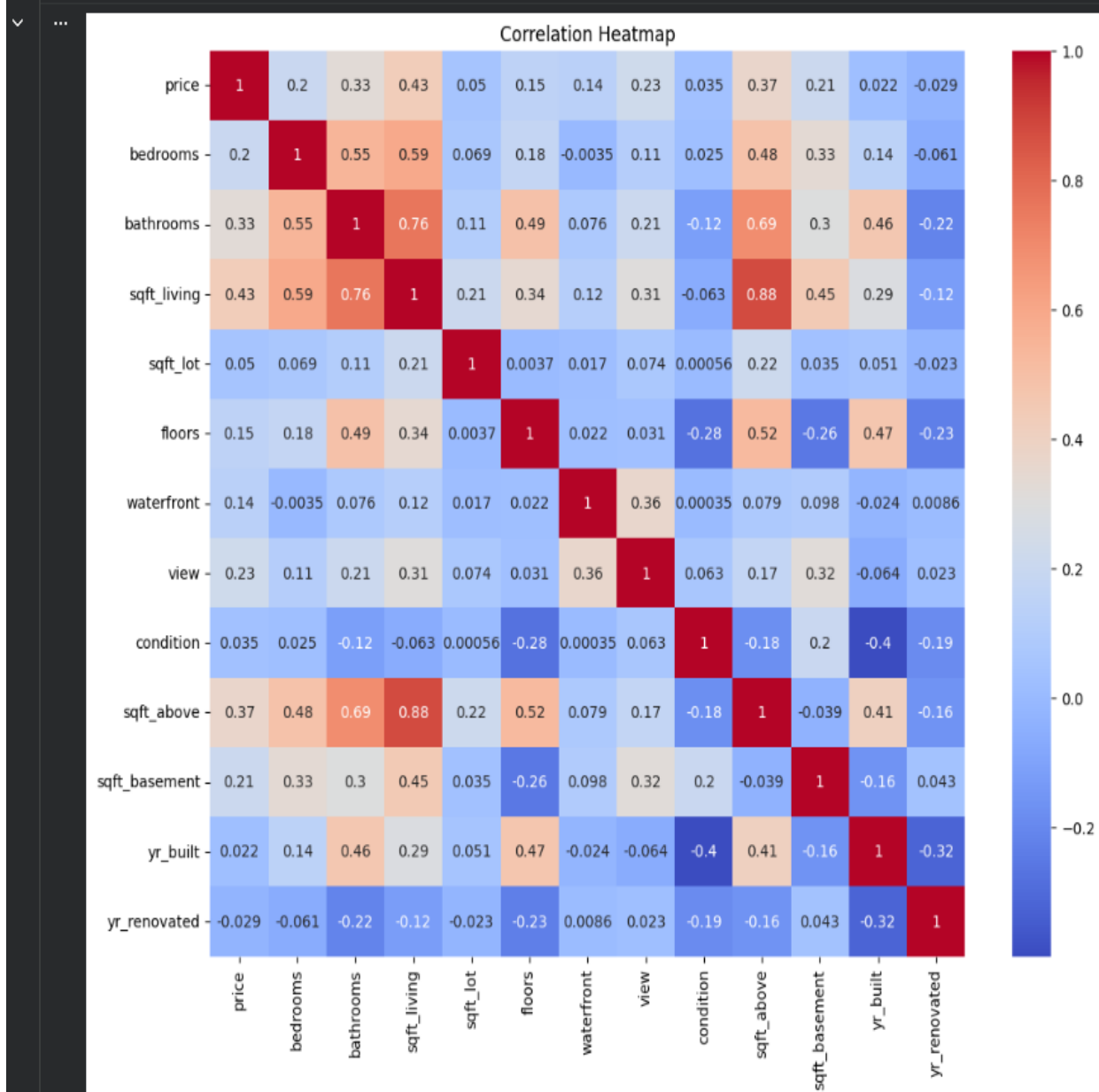



```
[ ] ▶ plt.figure(figsize=(12,10))

sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="coolwarm")

plt.title("Correlation Heatmap")

plt.show()
```



[]



```
df.isnull().sum()
```



0

date

0

price

0

bedrooms

0

bathrooms

0

sqft_living

0

sqft_lot

0

floors

0

waterfront

0

view

0

condition

0

sqft_above

0

sqft_basement

0

yr_built

0

yr_renovated

0

street

0

city

0

statezip

0

country

0

dtype: int64

5. Data Preprocessing

Several preprocessing steps were applied before model training.

Missing Values

Missing values were identified and handled appropriately to ensure clean input data.

Encoding

Categorical variables such as city, street, and country were converted into numerical values using encoding techniques.

Scaling

Feature scaling was applied where necessary to normalize numerical features.

Log Transformation

The target variable (Price) was transformed using a logarithmic transformation to reduce skewness and improve model performance.

6. Feature Engineering

Feature engineering was performed to improve the predictive performance of the models.

The following techniques were used:

- Label Encoding
- Log Transformation
- Feature Selection
- Train-Test Split

These transformations helped improve model accuracy while reducing noise in the data.

7. Machine Learning Models

Three regression algorithms were implemented and compared.

1. Linear Regression

A baseline regression algorithm used for comparison.

Advantages

- Simple
- Fast
- Easy to interpret

2. Random Forest Regressor

A tree-based ensemble learning algorithm that combines multiple decision trees.

Advantages

- High accuracy
- Handles nonlinear relationships
- Reduces overfitting

3. Gradient Boosting Regressor

A boosting algorithm that builds trees sequentially to improve prediction accuracy.

Advantages

- High predictive power
- Strong performance on structured datasets

8. Model Evaluation

The models were evaluated using:

- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)

Model	RMSE	MAE
Linear Regression	1.407601	0.430468
Random Forest	1.395156	0.331387
Gradient Boosting	1.396506	0.475364

Best Model

The **Random Forest Regressor** achieved the lowest RMSE and MAE, making it the best-performing model for this project.

[]



```
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

def evaluate_model(name, y_true, y_pred):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    return [name, rmse, mae]

results = []

results.append(evaluate_model("Linear Regression", y_test, lr_pred))
results.append(evaluate_model("Random Forest", y_test, rf_pred))
results.append(evaluate_model("Gradient Boosting", y_test, gb_pred))

results_df = pd.DataFrame(results, columns=["Model", "RMSE", "MAE"])

results_df.sort_values("RMSE")
```



Model

RMSE

MAE



1	Random Forest	1.395156	0.331387
2	Gradient Boosting	1.396506	0.475364
0	Linear Regression	1.407601	0.430468

9. Residual Analysis

Residual analysis was performed to evaluate prediction errors.

Residual Scatter Plot

The residual plot showed that most prediction errors were randomly distributed around zero, indicating that the model captured the relationship between features and house prices effectively.

(Insert Screenshot: Residual Plot)

Residual Distribution

The histogram of residuals showed an approximately normal distribution with a few outliers, suggesting good model performance

```
[ ] ▶ best_pred = rf_pred

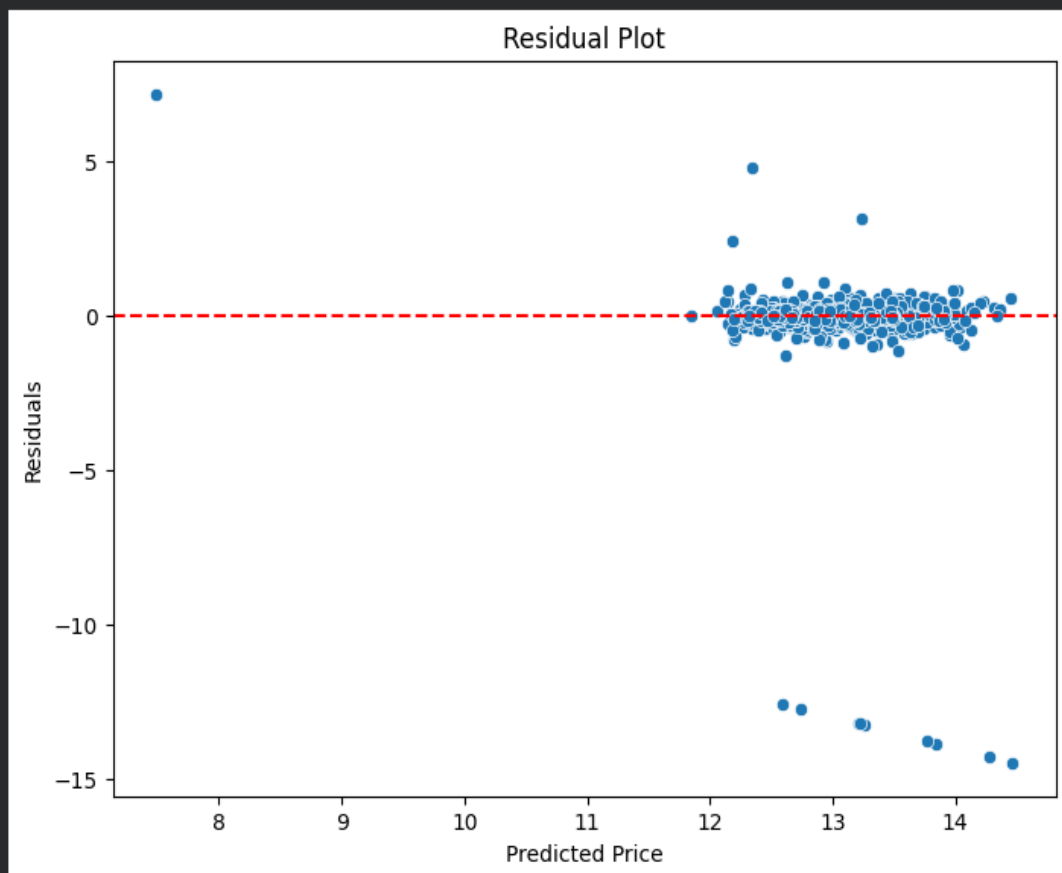
residuals = y_test - best_pred

plt.figure(figsize=(8,6))
sns.scatterplot(x=best_pred, y=residuals)

plt.axhline(0, color="red", linestyle="--")

plt.xlabel("Predicted Price")
plt.ylabel("Residuals")
plt.title("Residual Plot")

plt.show()
```



10. Prediction Example

The trained Random Forest model was saved using Joblib and used for making predictions.

Sample Prediction

```
import joblib
import numpy as np
model = joblib.load("house_price_model.pkl")
sample = X.iloc[[0]]
prediction = model.predict(sample)
print("Predicted House Price:", np.expml(prediction[0]))
```

Output

Predicted House Price:
310998.43

```
[ ] import joblib
import numpy as np

model = joblib.load("house_price_model.pkl")

sample = X.iloc[[0]]

prediction = model.predict(sample)

print("Predicted House Price:", np.expml(prediction[0]))

▼ Predicted House Price: 310998.4306661456
```

11. Conclusion

This project successfully developed a machine learning model for house price prediction using multiple regression algorithms.

The project followed a complete machine learning workflow including data preprocessing, exploratory data analysis, feature engineering, model training, evaluation, and deployment.

Among the three evaluated models, the **Random Forest Regressor** achieved the best performance with the lowest RMSE and MAE values. Residual analysis further confirmed the effectiveness of the model.

The saved model can be used to predict house prices for new properties and can be further improved using advanced hyperparameter tuning, feature selection, or ensemble learning techniques.

12. References

- Scikit-learn Documentation
- Pandas Documentation
- NumPy Documentation
- Matplotlib Documentation
- Seaborn Documentation
- Google Colab
- GitHub

Appendix

GitHub Repository

<https://github.com/ankitbungla20-student/House-Price-Prediction>

Google Colab Notebook

https://colab.research.google.com/drive/1vHaCLsh7_3tucLm3WqN3BUyGf4Olf58m?usp=sharing

